MyCodeSchool

Code review — andrei-backtracking

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-15 — sha a8e1fc0

Ce am verificat: fișa `exercitii-permutari.pdf` (8 probleme). Fiecare fișier `permutari-N.h` l-am compilat separat și l-am rulat cu datele de intrare din fișă, apoi am comparat ieșirea cu cea din fișă.

Problema	Fișierul tău	Rezultat
1. începe cu n	`permutari-1.h`	✅ `3 1 2` / `3 2 1`
2. se termină cu 1	`permutari-2.h`	🔴 `1 2 1` / `2 1 1` (B3)
3. poziții pare → valori pare	`permutari-3.h`	✅ 4/4 rânduri
4. fără punct fix	`permutari-4.h`	✅ 9/9 rânduri
5. fără consecutive alăturate	`permutari-5.h`	🔴 nu compilează (B4)
6. parități alternative	`permutari-6.h`	✅ 8/8 rânduri
7. 1 înaintea lui n	`permutari-7.h`	🔴 fără condiție (B5)
8. vecini apropiați	—	🔴 lipsește (B5)

Patru probleme sunt corecte (1, 3, 4 și 6). Funcția `valid(k)` o scrii bine: verifici întâi că valoarea nu se repetă, apoi adaugi condiția problemei, iar la 5 și 6 ai pus corect `k>1` înainte să te uiți la `x[k-1]`. Ce n-a mers ține de cum ai legat fișierele între ele și de faptul că nu ai verificat ieșirea cu cea din fișă.

🔴 Critice

B1 🔴 Proiectul nu compilează — `main.cpp:2`, `main.cpp:10`

CUM E ACUM:

```
main.cpp:10: error: use of undeclared identifier 'permutari2';
did you mean 'permutari6'?
```

`main.cpp` include `permutari-6.h`, dar apelează `permutari2()`, care există doar în `permutari-2.h`.

DE CE: compilatorul nu caută singur prin folder. Vede doar ce i-ai pus în față cu `#include`. Faptul că un fișier e adăugat în proiectul din Code::Blocks (`.cbp`) nu înseamnă că e compilat: un `.h` ajunge la compilator doar prin `#include`. Deci din cele 7 fișiere ale tale, la fiecare Build s-a compilat unul singur. Restul n-au fost compilate niciodată, și așa s-a ascuns B4.

FIX: un singur `main.cpp` care include fișierele și apelează problemele una după alta (după ce repari B2).

B2 🔴 Garda `#ifndef` cu minus nu e o gardă — `permutari-1.h:1-2` (+ același lucru în `permutari-2.h` … `permutari-7.h`, linia 1-2)

CUM E ACUM: am pus două fișiere în același program:

```
#include "permutari-1.h"
#include "permutari-3.h"
int main(){ permutari3(); }
```

```
error: use of undeclared identifier 'permutari3'
```

`permutari-3.h` a fost șters fără nicio eroare, iar compilatorul s-a plâns abia mai jos.

DE CE: un nume de macro poate avea litere, cifre și `_`, dar nu `-`. Când preprocesorul citește `#define PERMUTARI-1_H_INCLUDED`, numele se oprește la minus: macro-ul se cheamă `PERMUTARI`, iar `-1_H_INCLUDED` devine valoarea lui. Asta înseamnă warning-urile `extra tokens at end of #ifndef` și `requires whitespace after the macro name`. Toate cele 7 fișiere ajung să aibă aceeași gardă, `PERMUTARI`. Primul inclus o definește, iar al doilea vede `#ifndef PERMUTARI` fals și sare peste tot conținutul lui.

Și dacă garda ar fi corectă, tot n-ar merge: fiecare fișier declară din nou `n`, `x`, `valid` și `afisare`. Două fișiere în același program înseamnă `redefinition of 'n'`.

FIX: `_` în loc de `-` în gardă. Partea comună (`n`, `x`, `afisare`) rămâne doar în `permutari.h`. Fiecare problemă include `permutari.h` și își scrie doar `validN`, `backN` și `permutariN`, cu numele ei.

B3 🔴 Problema 2 pune 1 de două ori — `permutari-2.h:27`

CUM E ACUM: pentru n=3 iese `1 2 1` / `2 1 1`, în loc de `2 3 1` / `3 2 1`. Pentru n=4 iese `1 2 3 1`, `1 3 2 1` ...

DE CE: ideea e bună. Ai fixat `x[n]=1` și completezi doar primele `n-1` poziții. Dar `valid(k)` se uită doar în urmă, la pozițiile `1..k-1`. Pe `x[n]` nu-l vede niciodată, pentru că e în fața lui `k`, nu în urmă. Așa că `valid` nu are de unde să știe că 1 e deja folosit. Iar bucla `for(int v=1;v<n;v++)` îi oferă exact valoarea 1. Mai e o problemă: dacă 1 e luat, pe primele `n-1` poziții trebuie să intre `2..n`, iar bucla ta se oprește la `n-1`, deci lui `n` nu-i dă voie să intre niciodată. Asta e jumătatea care lipsește din ieșire: nu apare niciun 3.

FIX: valorile disponibile sunt `2..n`, deci `for(int v=2;v<=n;v++)`.

B4 🔴 Problema 5 nu compilează — `permutari-5.h:50`

CUM E ACUM:

```
permutari-5.h:50: error: use of undeclared identifier 'back';
did you mean 'back5'?
```

DE CE: ai redenumit funcția în `back5`, dar apelul din `permutari5()` a rămas `back(1)`. N-ai văzut eroarea pentru că fișierul n-a fost inclus niciodată (B1). Logica e corectă: am înlocuit doar `back(1)` cu `back5(1)` și ieșirea e exact cea din fișă, `2 4 1 3` / `3 1 4 2`.

FIX: `back5(1);`

B5 🔴 Problema 7 n-are condiția, iar problema 8 lipsește — `permutari-7.h:7-15`

CUM E ACUM: `permutari-7.h` e copia lui `permutari.h`. Pentru n=3 afișează toate cele 6 permutări, în loc de 3 (`1 2 3`, `1 3 2`, `2 1 3`). Problema 8 nu are niciun fișier. Commit-ul se cheamă „final tema joi".

DE CE: o copie a șablonului compilează și afișează ceva, așa că pare „gata". Ce o deosebește de rezolvare e doar condiția din `valid`, iar aici condiția lipsește. Singura verificare care prinde asta e să pui ieșirea ta lângă cea din fișă, rând cu rând: 6 rânduri față de 3 se văd imediat.

FIX: la 7, când pui `n` pe poziția `k`, 1 trebuie să fie deja printre `x[1..k-1]`. La 8, `abs(x[k]-x[k-1])<=2` pentru `k>1`.

Before / After

	Acum	Corect
B1	`#include "permutari-6.h"` + `permutari2();`	include fiecare problemă și apelează funcția ei
B2	`#ifndef PERMUTARI-1_H_INCLUDED`	`#ifndef PERMUTARI_1_H_INCLUDED`
B2	`int n, x[20];` în fiecare fișier	doar în `permutari.h`, inclus de toate
B3	`for(int v=1;v<n;v++)`	`for(int v=2;v<=n;v++)`
B4	`back(1);`	`back5(1);`
B5	`permutari-7.h` = copia șablonului	`valid` cu condiția „1 înaintea lui n"

🟡 Importante

- **M1** `permutari-1.h:44`: pentru n=1 nu afișează nimic (trebuia `1`). `back(2)` pornește de la `k=2`, iar `k==n` nu mai e adevărat niciodată. La problema 2 e la fel (`permutari-2.h:42`): pentru n=1 nu afișează nimic.
- **M2** Numele nu se potrivesc între fișiere: în `permutari-1.h` și `permutari-7.h` funcția e `permutari()`, în celelalte e `permutariN()`. În `main.cpp:9` ai comentat un `permutari1()` care nu există. O regulă, aplicată în toate 8 fișierele.
- **M3** `permutari-5.h:16`: `abs` fără `#include <cstdlib>`. Merge acum pentru că îl aduce `<iostream>` pe compilatorul tău, dar nu e garantat.

🟢 Cleanups

- **C1** ✅ Urmărirea de mână din `permutari.h:46-108` e corectă și se potrivește cu ieșirea programului (`1 2 3` … `3 2 1`). O singură scăpare, la linia 84: `valid(2)` trebuia să fie `valid(3)`.
- **C2** `permutari.h:4-5`: `using namespace std;` de două ori.
- **C3** `permutari-2.h:5`: `x[10]`, în celelalte e `x[20]`.
- **C4** Commit-urile apar ca `KERNELOS-PC\Administrator`. Setează `git config --global user.name` și `user.email`.

Ce urmează

1. B2 întâi: garda cu `_` și `n` / `x` / `afisare` doar în `permutari.h`.
2. B1: un `main.cpp` care le include pe toate și le apelează pe rând. Dacă compilează, ai rezolvat și B4.
3. B3, apoi problemele 7 și 8.
4. Pentru fiecare problemă: rulezi cu datele din fișă și compari rând cu rând.

Întrebări

1. De ce a compilat proiectul tău cu `permutari-5.h` în el, deși `permutari-5.h` are o eroare?
2. La problema 2, `valid(k)` se uită la `x[1]` … `x[k-1]`. Pentru `k=1`, la ce poziții se uită? Și de ce nu vede niciodată `x[n]`?
3. Dacă scrii `#define A-B 5`, cum se cheamă macro-ul și ce valoare are?